PROGRAMMING WITH PYTHON II

Sessions 15 & 16

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Index

1. Intelligence in the Backend
2. Toolset: Scikit-learn vs. PyTorch
3. Model Persistence with Joblib
4. Including an Efficient Prediction Endpoint
5. Introduction to Streamlit

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Intelligence in the Backend

- **From CRUD to Intelligence:**
  - Backends typically use CRUD operations to store and manage raw data.
  - Intelligence is added when we extract patterns from this data to make predictions.
- **The Role of FastAPI:**
  - FastAPI provides a production-ready framework to expose these patterns via prediction endpoints.
  - It uses Pydantic for the "data parts," ensuring that input data is validated before reaching the ML model, which reduces errors by ~40%.

Toolset: Scikit-learn vs. PyTorch

- **Why Persist?** We train models once and serialize them into a stream of bytes to be stored on disk and loaded by the backend later.
  - Scikit-learn is currently the most widely used toolset for traditional Machine Learning.
  - It is ideal for tabular data and standard algorithms (Random Forests, SVMs).
- **PyTorch**
  - One of the top two deep learning frameworks (alongside TensorFlow).
  - Designed for neural networks and compute-intensive tasks.
- **Key Difference**
  - Scikit-learn focuses on general-purpose ML, while PyTorch is a powerful framework for deep learning.
  - Both leverage vectorised operations, which can provide speedups of up to 500 times compared to explicit Python loops.

scikit-learn vs PyTorch

scikit-learn

Data → Model → Predict?

→ model.fit(), model.predict()

vs

PyTorch

Data → Tensor →

build model → training loop

vs

Tensor

Forward Pass

Backward Pass

Compute Loss

Update Weights

The Key Difference

Easy Mode

→ Automatic
→ Simple
→ Done!

Hard Mode

→ Manual
→ Flexible
→ Full Control

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Visualizing the Key Difference

scikit-learn

↓

`model.fit(X, y)`

↓

Library trains internally

You do not control training.

PyTorch

↓

Define Model

↓

Training Loop

↓

Forward pass

↓

Optimizer update

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Model Persistence with Joblib

- **Why Persist?**
  - We train models once and serialize them into a stream of bytes to be stored on disk and loaded by the backend later.
- **Using Joblib/Pickle:**
  - Joblib (an evolution of Pickle) lets you share Python objects between programs you control.
- **Security Warning**
  - Scikit-learn focuses on general-purpose ML, while PyTorch is a powerful framework for deep learning.
  - Both leverage vectorised operations, which can provide speedups of up to 500 times compared to explicit Python loops.

Including an Efficient Prediction Endpoint

- The Concurrency Challenge
  - ML predictions are often CPU-bound. In an async environment like FastAPI, heavy computation can block the event loop.
- Handling Blocking Calls
  - If a prediction takes significant time, it will pause the entire program for all users.
- The Solution
  - Use asyncio.to_thread (Python 3.9+) to run the prediction in a separate thread.
  - For multi-core scaling, use a ProcessPoolExecutor to bypass the Global Interpreter Lock (GIL).
  - External Task Queues (like Celery) are recommended for long-running jobs to keep the API responsive.

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Introduction to Streamlit

- **Frontend for ML**
  - Streamlit is essentially a "cheat code" for data scientists and Python developers. It is an open-source Python library that allows you to turn data scripts into shareable web apps in minutes—without requiring any knowledge of HTML, CSS, or JavaScript.
- **What it provides**
  - Streamlit allows you to build an interactive dashboard to interact with your FastAPI endpoint.
- **Integration**
  - Project Jupyter illustrates how Python/JavaScript applications can be integrated via messaging.
  - Use Streamlit to create sliders and inputs that send data to your FastAPI prediction endpoint and display the results visually.

Lets practice

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Bibliography

- Fluent python (Luciano Ramalho)
  - Chapter 3: Functions
  - Chapter 6: Object References, Mutability, and Recycling
  - Chapter 7: Functions as First-Class Objects
  - Chapter 9: Decorators and Closures
- Effective communication (Brett Slatkin)
  - Chapters 3 and 4: Comprehensions and Generators
  - Chapter 5: Type hinting
  - Chapter 7: Asynchronous I/O
  - Chapter 10: Virtual environments
- Asynchronous I/O: Python documentation
  - https://docs.python.org/3/library/asyncio-task.html

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY